

CS 234 Advanced Networks Programming Assignment 1

1 Mp4Client Model Parsing

At first, since there is no `model` attribution inside the MPD, we should add it to the MPD data structure. We can find MPD's head file in the following directory:

```
~/gpac/include/gpac/internal/mpd.h
```

In this file, we can see a structure called `GF_MPD_AdaptationSet`, which contains all the attributions in the adaptation set. Since the `model` is represented by a number, we can add it to the structure as follows(in actuality it is already here):

```
int model;
```

After studying the structure of the `gpac`, we can find the file that `Mp4Client` uses to parse MPD is in the following directory:

```
~/gpac/src/media_tools/mpd.c
```

When digging into the `mpd.c` file, we can see that this program parse adaptation set in this function:

```
static GF_Err gf_mpd_parse_adaptation_set(GF_MPD *mpd,  
                                           GF_List *container, GF_XMLNode *root)
```

By observing this function, we can see that it use `strcmp` to get the attribution name, then set the corresponding attribution with the value of that attribution name. For example, if you want to parse `maxHeight`, it will do it by:

```
else if (!strcmp(att->name, "minHeight"))  
    set->min_height = gf_mpd_parse_int(att->value);
```

If you want to parse `segmentAlignment`, it will do it by:

```
else if (!strcmp(att->name, "segmentAlignment"))  
    set->segment_alignment = gf_mpd_parse_bool(att->value);
```

So now we get the pattern that the `mpd.c` parses XML files. If we want to parse `model`, we can easily imitate the snippets above to accomplish it. We can add this snippet inside the method:

```
else if (!strcmp(att->name, "model"))
    set->model = gf_mpd_parse_int(att->value);
```

To print the `model` in the adaptation set, we can find another function inside the `mpd.c` file:

```
static void gf_mpd_print_adaptation_set
(GF_MPD_AdaptationSet const * const as, FILE *out)
```

Inside this function, we can see that if you want to print the attribution of `maxWidth`, the function will do it by:

```
if (as->max_width)
    fprintf(out, " maxWidth=\"%d\"", as->max_width);
```

By imitation, we can print the attribution of `model` by:

```
if (as->model) fprintf(out, " model=\"%d\"", as->model);
```

To print the `model` when the new segment has been downloaded, we should implement DASH client, which is in:

```
~/gpac/src/media_tools/dash_client.c
```

Inside this file, we can find a function called:

```
static void dash_do_rate_adaptation(GF_DashClient *dash,
    GF_DASH_Group *group, int adaptation_set_num)
```

At this stage, we can implement a few lines of code to print the final result:

```
printf("adaptation_set_num: %d, model: %d\n",
    adaptation_set_num, group->adaptation_set->model);
```

2 Multiple Periods MPD Support

Can the current MP4Client handle multiple Periods in an MPD file? Well, in my opinion, the answer is positive. At first, we can look into the `mpd.h` file. Because this head file defined the structure of the MPD, which will be applied in `mpd.c`, `dash_client.c` and so on. The `mpd.h` file is in this directory:

```
~/gpac/include/gpac/internal/mpd.h
```

In this file, we can see a structure called `GF_MPD`. Inside this structure, it has an attribution called `periods`, which stands for a list of periods:

```
GF_List *periods;
```

At this point, we've known that a MPD can consist multiple periods.

Now we can go and see how `dash_client.c` support multiple period MPD. We can find the `dash_client.c` at:

```
~/gpac/src/media_tools/dash_client.c
```

This program uses a function called `gf_list_get` to handle the `period` list. The function `gf_list_get` is imported from:

```
~/gpac/include/gpac/list.h
```

In this head file, we can see its usage clearly. This function is defined as follows:

```
void *gf_list_get(GF_List *ptr, u32 position);
```

It helps to get an item from the list given its position. So back to the `dash_client.c` file, for example, there is a function called `gf_dash_seek_groups`:

```
static void gf_dash_seek_groups(GF_DashClient *dash,
    Double seek_time, Bool is_dynamic)
```

Inside this function, we can look at the following code snippet:

```
for (i=0; i<dash->active_period_index; i++) {
    GF_MPD_Period *period = gf_list_get(dash->mpd->periods,
        dash->active_period_index-1);
    dur += period->duration/1000.0;
}
```

It add duration of all active periods inside a MPD up. It proved that the client can handle multiple periods MPD.

3 Assign Tile Quality Levels

In this section, we will focus on the following file:

```
~/gpac/src/media_tools/dash_client.c
```

In this file, we are focusing on modifying the following function:

```
static void dash_do_rate_adaptation(GF_DashClient *dash,
    GF_DASH_Group *group, int adaptation_set_num)
```

As we can see from the instruction of this assignment, different value of models refers to different representations.

At first, in this function, we get the initial representation by getting the active representation inside the group:

```
rep = gf_list_get(group->adaptation_set->representations,
    group->active_rep_index);
```

As we can observe from the source code, the function gets new representation by assigning a new index of representation:

```
new_rep = gf_list_get(group->adaptation_set->representations,
    (u32)new_index);
```

So, to get different representations, we only need to change the value of `new_index`. If we want to get the representation of specific model, we only need to change the `new_index` to the model value.

As we can see, the `new_index` is initialized by:

```
new_index = 0;
```

So the default model is 0. In this case, we only need to change the `new_index` to `model - 1` to switch the representation:

```
new_index = group->adaptation_set->model - 1;
```

4 Call Time for Downloading Segments

In a 3X3 tiled segment, it would be called at least 9 times. In a 5X5 tiled segment, it would be called at least 25 times.